

CT500 Firmware User Guide

Version 2.0

Shmt Co., Ltd.

REVISION HISTORY PAGE

Issue	Rev No.	Originator	Details of Change	Date
1	0.1	DJKIM	Initial Version	2007-02-14
2	0.2	DJKIM	Add HOWTO porting	2007-02-23
3	0.3	DJKIM	Modify User Manual	2007-03-01
4	0.4	DJKIM	Modify HOWTO contents	2007-03-13
5	1.0	DJKIM	Modify HOWTO build execute file	2007-03-26
6	2.0	DJKIM	Modify Directory Analysis contents	2007-06-07

Confidential

Technical documents for Firmware

Copyright © 2007 Shmt Limited. All rights reserved.

Confidentiality Status

This document is **NOT OPEN ACCESS**. The distribution is restricted to certified company.
This document is **CONFIDENTIAL**.

Feedback on this document

If you have any comments on this document, please contact your channel to our company.

Shmt Web address

<http://www.shmt.com>

Confidential

Contents

1. INTRODUCTION	7
2. OverView	8
3. SW Hierarchy	12
4. Directory Analysis	14
4.1. main	16
4.1.1. app	16
4.1.2. bootloader	16
4.1.3. startup	16
4.2. sys	16
4.3. peri	17
4.4. inc	17
4.5. lib	18
4.5.1. drivers	18
4.5.2. basic peripheral	18
5. HOWTO build execute file	19
5.1. ADS v1.2 Using	19
5.1.1. Firmware Project Structure	19
6. HOWTO Porting	22
6.1. Memory map	22
6.2. System setting	23
6.3. Source modification	23
APPENDIX	25
A. ASSEMBLER DIRECTIVES	25
C. CACHE AND MMU INSTRUCTION	27

Figures

[Fig - 1] 펌웨어 S/W 계층 구조.....	12
[Fig - 2] Pre-level driver 디렉토리 구조.....	13
[Fig - 3] F/W 디렉토리 구조	15
[Fig - 4] ADSv1.2 프로젝트 구조.....	19
[Fig - 5] Debug 모드에서 실행 파일 생성 시	20
[Fig - 6] axd를 사용한 디버깅 시.....	20
[Fig - 7] 릴리즈 모드에서 실행 파일 생성 시.....	21
[Fig - 8] 롬에 물레이터 사용 시.....	21
[Fig - 9] ROM base memory map	23
[Fig - 10] DDR-SDRAM 기반 메모리 맵	23
[Fig - 11] 도메인 액세스 컨트롤 레지스터 cp15:c3 의 포맷.....	27
[Fig - 12] MMU 안에 위치한 cp15:c1 레지스터 컨트롤 비트	27
[Fig - 13] 고속 문맥 전환 레지스터 cp15 레지스터 13	28
[Fig - 14] 변환 테이블 베이스 주소 cp15 레지스터	28
[Fig - 15] Way와 세트 인덱스 주소 지정 방식을 사용한 cache clean시 cp15:c7 레지스터 Rd의 형식	28
[Fig - 16] 주메모리 안에서 원래 값으로 참조된 cache line을 clean하고 flush할 때의 cp15:c7 레지스터 포맷.....	29
[Fig - 17] Lock down bit 사용 cache lock했을 때 cp15:c9 레지스터 포맷	29

Tables

[Table - 1] TLB 플러시하기 위한 cp15:c8 명령어.....	27
[Table - 2] TLB 락다운 레지스터를 액세스하는 명령	27
[Table - 3] MMU 제어 컨트롤 레지스터 cp15:c1 의 비트필드	28
[Table - 4] Cache flush 명령어	28
[Table - 5] Way 와 세트 인덱스 주소 지정 방식을 사용한 cache line clean & flush cp15:c7 명령어	28
[Table - 6] 주메모리 안에서 위치를 참조하여 cache line clean 하고 flush 하는 명령어	29
[Table - 7] Lock down bit 를 사용하여 cache 를 lock 하기 위한 cp15:c9 명령어.....	29

1. INTRODUCTION

본 문서는 가라오케 시스템을 위한 low-cost 솔루션 SOC로 개발된 CT-500 의 펌웨어에 대한 내용을 다룬다. H/W 설계 과정 및 FPGA 검증 과정에서 사용되는 펌웨어의 내용과 디렉토리 및 S/W 계층 구조가 설명되었다.

Confidential

2. OverView

CT500 은 ARM926EJ-S을 내장하고 있으며 다양한 어플리케이션에 효율적으로 대응할 수 있는 SOC 칩으로 다음과 같은 특징을 갖는다.

- ARM926EJ core

Normal operation mode : 100MHz

2x operation mode : 200MHz

- Video Encoder

NTSC/PAL display

One DAC for CVBS, two times over-sampled with 10 bit resolution.

Internal Color Bar Generator.

Cross color reduction filter.

- Internal PSRAM

4 MBits

Optional configuration

- UART 4 Channel

UART 32 depth FIFO 내장한 버전 2 채널

UART 8 depth FIFO 내장한 버전 2 채널

- Graphic Engine

NTSC/PAL output mode : 704 x 480i / 704 x 576i (max)

VGA output mode : 640 x 480p(VGA), 320x240p(QVGA)

4pages

- Wide(1440x576) x 2page : no Palette

- VGA (640x576) x 2page : Palette 256 color/page

Overlay & α-blending : 3pages(Wide 1 page + VGA 2pages).

Support chroma keying : Color selectable.

Line Up-scaling : 480 -> 576 (NTSC -> PAL).

Up-scaling support function

- For helping NTSC->PAL conversion.

- Insert 1 horizontal line, copied from the 5th line, each 5 lines.

- No line filter required.

Configurable Outputs : RGB(565) & Sync/ITU-R BT.601(16bit)/ITU-R BT.656(8bit).

Internal Color Bar Generator.

Color Space Conversion.

- DDR Controller

128/256/512 Mbit

- I2C

Variable clock rate : 400kHz max.

- I2S

Configurable Master/Slave.

ADC(receive) & DAC(transmit) IF for audio Codec.

IF mode : Standard I2S, MSB(=Left) justified mode(configurable).

8/16/24/32bit Word lengths.

16k/32k/44.1k/48kHz Sampling Frequency.

Each 16 word FIFO entries for Transmit & Receive.

- Timer

- Enhanced-DMA

- Graphic DMA

- SPI

SPI clock speed up to 12MHz.

Ch-1 : For transferring MP3 bit-stream.

160Byte/10mS @128k Sampling.

Support DMA.

Ch-2 : For transferring commands.

Data Length(max) : 64bit(8byte).

No needs DMA.

- UART

Special UART (2ch)

Baudrate : 115200 max

Tx 32bytes & Rx 32bytes FIFOs.

31.25k baudrate support for MIDI.

Standard UART (2ch)

Baudrate : 115200 max

Tx 8bytes & Rx 8bytes FIFOs.

- **Multi-boot**

For support wide range application

- **Sound Engine**

Mask rom for Sound Engine : 32 / 64 / 128 MBits

Internal SRAM for Sound Engine : 1 MBits + 24K bits

Operation Clock : 100MHz.

(128K + 3072)Byte SRAM

Sound effect & MIC echo memory.

512Byte ROM.

Audio CODEC : Excluded.

Wave Playback & Recorder Interface.

1024 point FFT with I2S receiver.

4ch I2S Master interface : 2ch ADC in & 2ch DAC out .

PCM Rx/Tx interface for Record & Play.

- **NAND Flash memory controller**

32M ~ 2G Byte.

512/256 byte, 1-bit ECC.

Command FIFO/Data FIFO.

Support small/large block.

Support SLC/MLC.

- **USB 2.0**

USB 1.1/2.0 compliant.

PHY included.

- **External Video Input**

Digital Video Decoder Interface : ITU-R BT.656(8bit).

Supports Interlace / Non-interlace Mode.

Color Space Conversion.

Memory Buffering.

Support Frame Capture.

- Extra Asynchronous Chip select signal for reservation : 4 (8/16bit).

Async IF – ex) ROM/Nor-flash/SRAM IF.

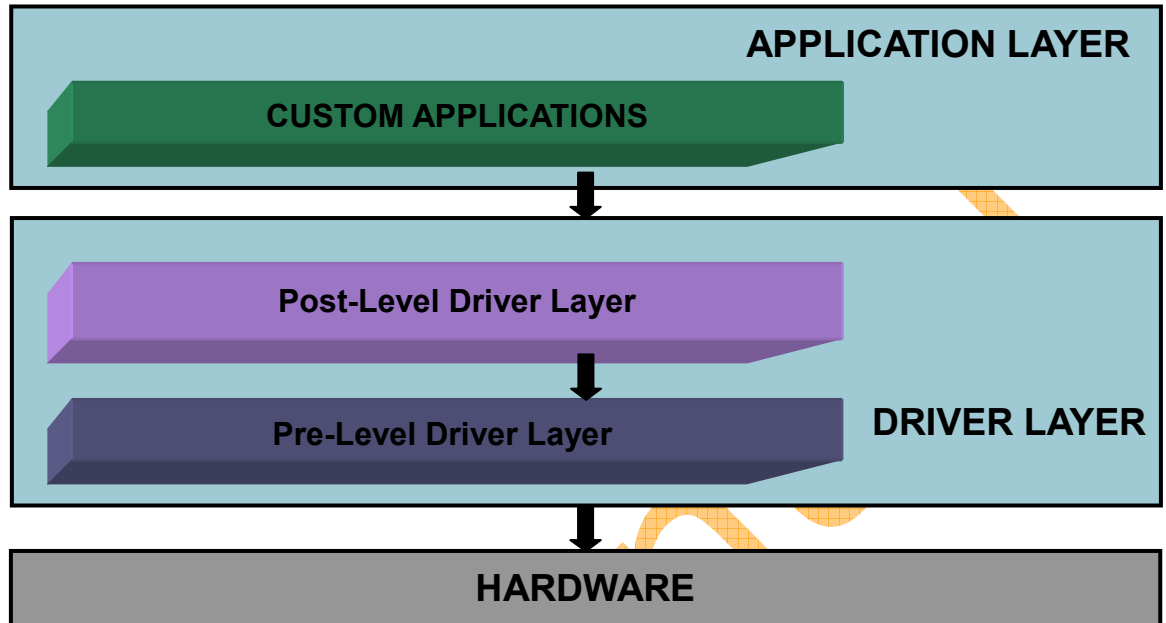
Wait configurable.

Upto 128M Byte.

Confidential

3. SW Hierarchy

다음 그림은 CT-500의 펌웨어 S/W 계층 구조를 나타낸다.



[Fig - 1] 펌웨어 S/W 계층 구조

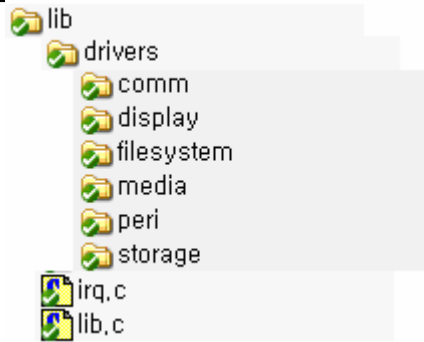
CT-500의 H/W를 펌웨어로 구성함에 있어 S/W 펌웨어 계층은 세가지로 구분하였다.

1) Pre-level driver layer

기본 peri(Memory, Timer/PWM, WDT, GPIO, VIC, ADC, Power management etc.)와 통신(UART, I2C, I2S, SPI etc), 디스플레이(LCD, Video Encoder), 미디어(AUDIO), 스토리지(NAND)의 레지스터 레벨 동작을 조작한다.

기본 peri에 대한 pre-level driver 함수는 lib/, lib/drivers/peri/ 디렉토리에 peri 모듈 별 파일로 구성이 되어 있으며, 함수명은 `smtfuncName()`로 지정된다.

통신, 디스플레이, 미디어, 스토리지의 모듈 별 파일 구성도 이와 비슷하다. Pre-level driver 함수들은 `module_pre_drv.c` 파일로 존재하며 다음은 pre-level driver 함수의 디렉토리 구조를 나타낸다.



[Fig - 2] Pre-level driver 디렉토리 구조

2) Post-level drivers layer

Pre-level driver layer보다 상위 드라이버 계층으로 pre-driver 함수를 호출하여 시나리오적인 관점의 동작을 조작한다. Pre-driver 함수의 경우 레지스터의 셋팅과 상태 체크 등 레지스터 레벨의 함수인 반면 post-driver 함수는 pre-driver 함수를 사용하여 읽기/쓰기 등과 같은 시나리오 동작과 관련된다.

Post-level driver 함수의 경우 pre-level driver 함수와 달리 함수명이 smt2 로 시작한다. Post-level driver 함수들은 pre-level driver 함수들과 같은 디렉토리에 module_post_drv.c 파일로 존재한다.

4. Directory Analysis

상화에서 제공하는 F/W 프로그램의 디렉토리는 다음과 같다.

1) inc/ 디렉토리

Driver 함수들의 선언과 사용되는 매크로, 전역 변수들을 담은 헤더 파일들이 있다.

drivers/

comm./

통신과 관련된 디바이스들(I2C, I2S, SPI, UART)의 드라이버 헤더 파일들이 있다.

display/

이미지 디스플레이 디바이스(LCD, Video Encoder, VIF)의 드라이버 헤더 파일들이 있다.

storage/

Storage (NAND, SD/MMC, SRAM, DDR-SDRAM) 디바이스의 드라이버 헤더파일이 있다.

fw_drv_err/

Post-level driver 함수의 에러 리턴을 위한 enum 정의 헤더파일이 있다.

commonmacro.h

- CT-500 F/W에서 사용할 수 있는 일반적인 매크로를 정의한다.

global.h

- CT-500 F/W에서 사용할 수 있는 전역 변수를 정의한다.

irq.h

- VIC를 위한 매크로와 함수 선언을 포함한다.

lib.h

- 기본 peripheral의 매크로 정의와 함수 선언을 포함한다.

structdef.h

- CT-500 F/W에서 사용되는 데이터 타입을 정의한다.

2) lib/ 디렉토리

Peripheral의 드라이버 함수들이 정의되어 있다.

drivers/

CT-500 에서 사용한 pre/post-level driver 함수들이 정의되어 있다.

comm./

I2C, I2S, SPI의 pre-level driver 함수가, UART 의 pre/post-level driver 함수가 정의되어 있다.

display/

LCD, Video Encoder, VIF의 pre-level driver 함수가 정의되어 있다.

storage/

NAND, SD/MMC, SRAM, DDR-SDRAM의 pre-level driver 함수가 정의되어 있다.

irq.c

VIC 관련 pre/post-level driver 함수가 정의되어 있다.

lib.c

기본 peripheral의 pre-level driver 함수가 정의되어 있다.

3) main/ 디렉토리

app/

유저의 F/W 소스 및 어플리케이션 소스가 존재한다.

bootloader/

NAND 부팅을 지원하기 위한 부트로더 소스가 존재한다.

startup/

펌웨어 main 함수로 진입하기 위해 필요한 ARM926 의 스타트 코드와 메모리 설정을 포함한다. (ADSv1.2 사용 할 경우 - [5.2 ADS v1.2 Using](#) 참조)

main.c

F/W 코드의 메인 함수이다.

mmu.c

ARM926EJ-S의 mmu 셋팅을 위한 함수들이 있다.

regview_v2.c

디버거 장비를 통한 디버깅 시 CT-500 의 레지스터 맵을 watch 할 수 있도록 레지스터 구조체가 정의되어 있다.

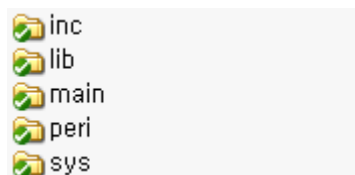
4) peri/ 디렉토리

각 peri 별 FPGA 테스트 파일과 헤더 파일이 존재한다.

5) sys/ 디렉토리

CT-500 의 시스템 레벨에서의 설정이 이루어진다. 메모리 사이즈, 클럭 설정, 메모리 버스 폭등을 설정한다.

[Fig- 4]는 디렉토리 구조를 나타낸다.



[Fig - 3] F/W 디렉토리 구조

Driver 함수에 대한 자세한 사항은 programming guide를 참조하기 바라며, 여기서는 스타트업 코드와 상화마이크로텍에서 제공하는 F/W 코드에서 공통적으로 사용하는 시스템 설정 관련 매크로 및 데이터 타입 정의에 대해서 설명하기로 한다.

4.1. main

4.1.1. app

CT-500 의 F/W 어플리케이션 소스가 있다.

4.1.2. bootloader

NAND 부팅에 사용되는 부트 로더가 있다.

4.1.3. startup

Startup 디렉토리의 파일들은 처음 리셋 후 CT-500 의 동작 시 main() 함수로 진입하기 전 이루어지는 초기화 함수들이 포함된다.

다음은 startup 디렉토리에 있는 파일들에 대한 설명이다.

1) Startup.s

ARM926EJ core startup 소스 파일이다.

2) smt926addr.inc

Startup.s에서 사용되는 메모리 बैं크 설정 변수의 값을 각 बैं크 별로 설정한다. 또한 버스, pll, 클럭 설정 변수의 값을 설정한다.

3) asmlib.s

ARM926EJ-S core의 irq 모드를 인에이블/디스에이블하는 어셈블리 함수가 있다.

4) mmufunc.s

ARM926EJ-S core의 MMU I/D-TLB의 클리어 어셈블리 함수가 있다.

4.2. sys

sys 디렉토리의 파일들은 CT-500 의 펌웨어 동작을 위한 시스템 관련 설정 헤더 파일들이다.

1) sysinc.h

Sys 디렉토리의 모든 헤더 파일을 포함한다.

2) sysdatadef.h

CT-500 의 펌웨어에서 사용되는 데이터 타입 을 정의한다.

3) sysreg.h

CT-500 의 레지스터 맵, 각 레지스터 별로 할당된 비트 마스크를 정의한다.

4) syscfg.h

CT-500 의 부트 모드, 플래쉬 메모리 사이즈등 시스템 관련 설정을 정의한다.

4.3. peri

peri 디렉토리의 파일들은 각 H/W 모듈 테스트 함수가 있는 파일들이다. 다음에서 설명하는 각 H/W 모듈들의 설명은 알파벳 순서에 따르도록 한다.

1) DMA

E-DMA(Enhanced)와 G-DMA(Graphic-DMA)로 구성되어 있으며, E-DMA는 memory to I/O, I/O to memory, memory to memory의 데이터 복사를 가능하게 한다.

G-DMA는 메모리에서 일정한 2 차원 영역을 다른 메모리 어드레스의 2 차원 영역으로 복사하는 일을 수행한다.

2) GPIO

3) TIMER

Timer는 interval, match & overflow 모드를 지원한다.

4) WDT

5) VIC

6) Communication Modules

가) I2C

나) I2S

다) SPI

마) UART

7) Display Modules

가) LCD

나) Video Encoder

8) Storage

가) Nand Flash

나) SD/MMC

다) SRAM

라) DDR-SDRAM

4.4. inc

CT-500 의 pre/post-level driver에서 함수 구현을 위해 사용하는 헤더파일이 있다. Driver

함수들을 위해서 서브 디렉토리 별로 pre/post-level driver 헤더파일들이 있다. 또한 기본 peripheral들을 위한 헤더파일(lib.h, irq.h)과 common 매크로 및 기본 peripheral의 프로그램 구조체가 정의된 헤더파일(commonmacro.h, structdef.h)가 있다.

4.5. lib

CT-500 의 pre/post-level driver 함수가 구현되어있다. Driver의 각 디렉토리 별로 pre/post-level driver 함수가, 기본 peripheral의 pre/post-level driver 함수는 irq.c, lib.c 파일에 구현되어 있다.

4.5.1. drivers

디바이스의 종류에 따라 comm.(I2C, I2S, UART), display(LCD, Video Encoder, VIF), storage(NAND, SD/MMC, SRAM, DDR-SDRAM)로 구분되며 각 디바이스 별로 pre-level 또는 post-level driver 함수들이 구현되어 있다.

4.5.2. basic peripheral

Timer, WDT, GPIO, VIC 등과 같은 기본적인 peri들의 pre-level 또는 post-level driver 함수들은 irq.c, lib.c 파일에 구현되어 있다.

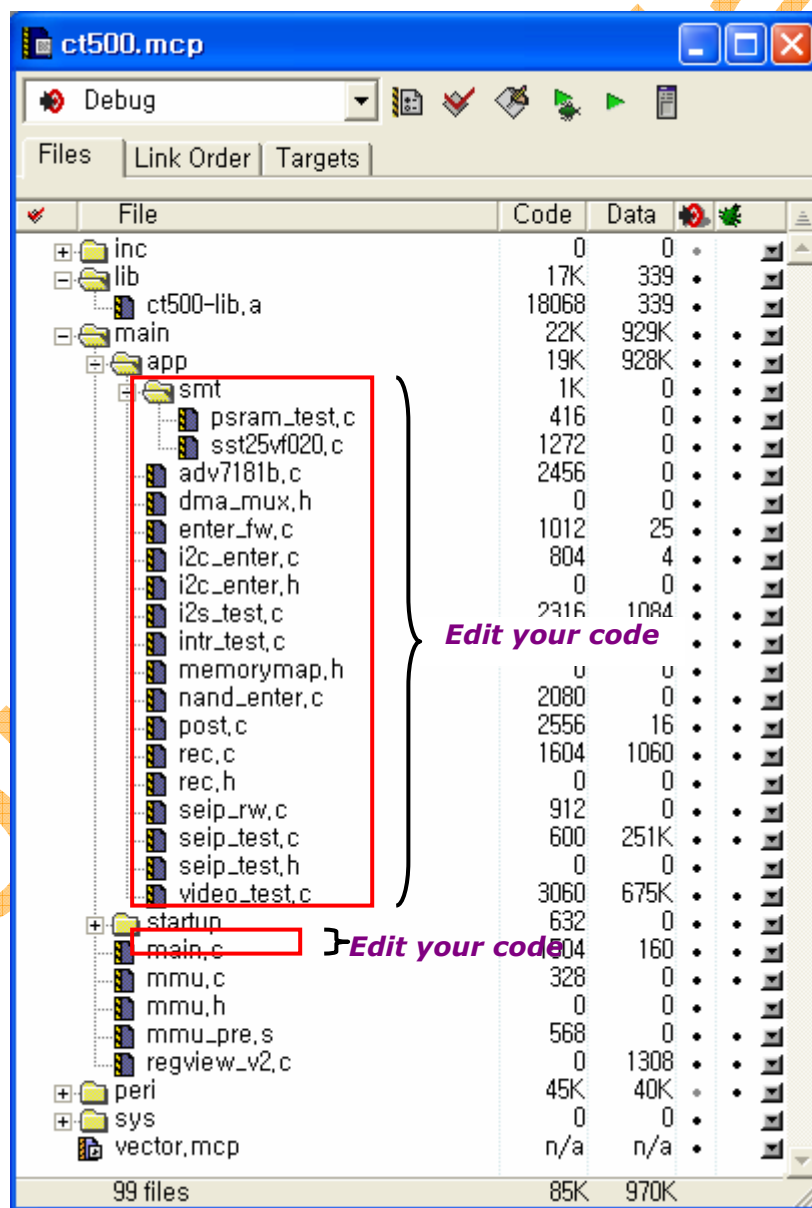
5. HOWTO build execute file

F/W의 실행 파일 생성은 windows상에서 ADSv1.2 IDE를 사용하여 생성할 수 있도록 구성되어 있다.

5.1. ADS v1.2 Using

5.1.1. Firmware Project Structure

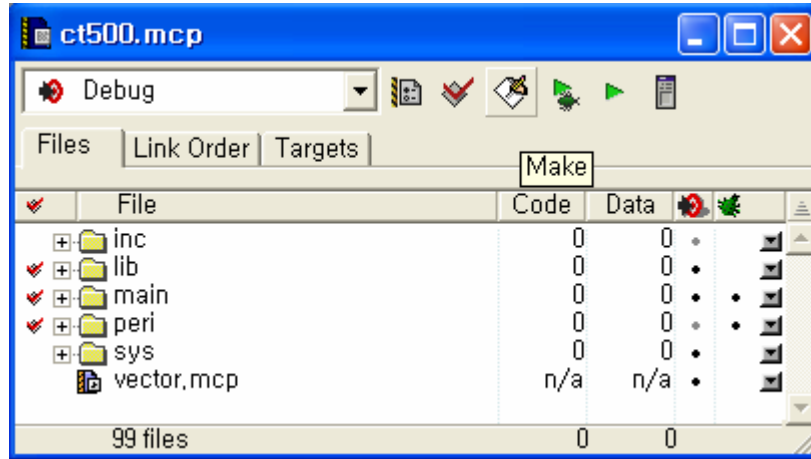
다음은 ADSv1.2 를 사용하여 구성한 CT-500 의 펌웨어 프로젝트 구조이다.



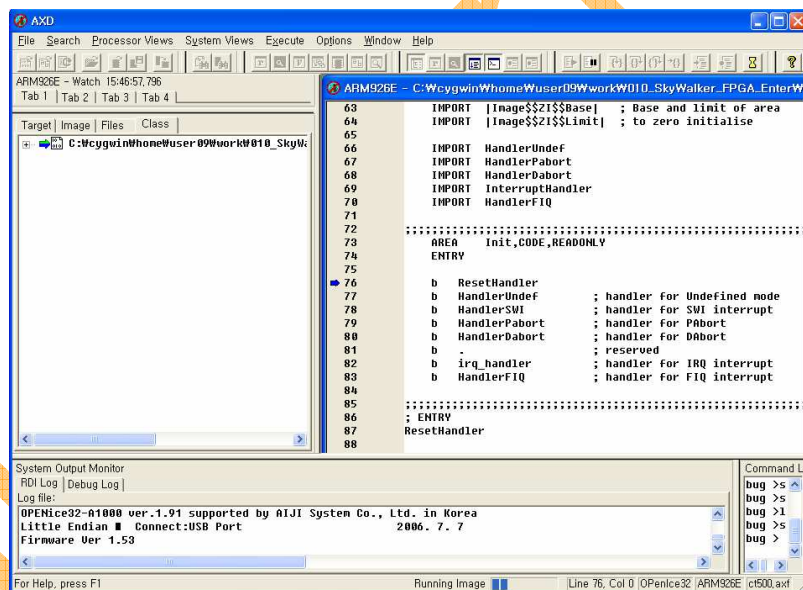
[Fig - 4] ADSv1.2 프로젝트 구조

응용프로그램 개발 시 a1000 과 같은 디버거 장비를 사용할 경우는 CT500 프로젝트

(ct500.mcp)에서 디버그 모드상에서 실행 파일을 생성하고 디버거 스크립트를 활용하여 axd 또는 aiji의 디버거 프로그램 spider를 사용하면 된다. [Fig - 6]과 [Fig - 7]은 실행 파일 생성과 디버거 사용 시 axd 상에서의 디버깅을 보여준다.

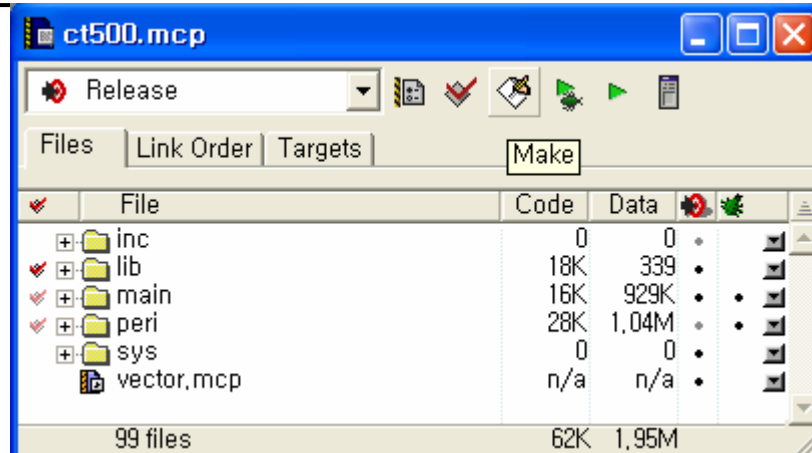


[Fig - 5] Debug 모드에서 실행 파일 생성 시

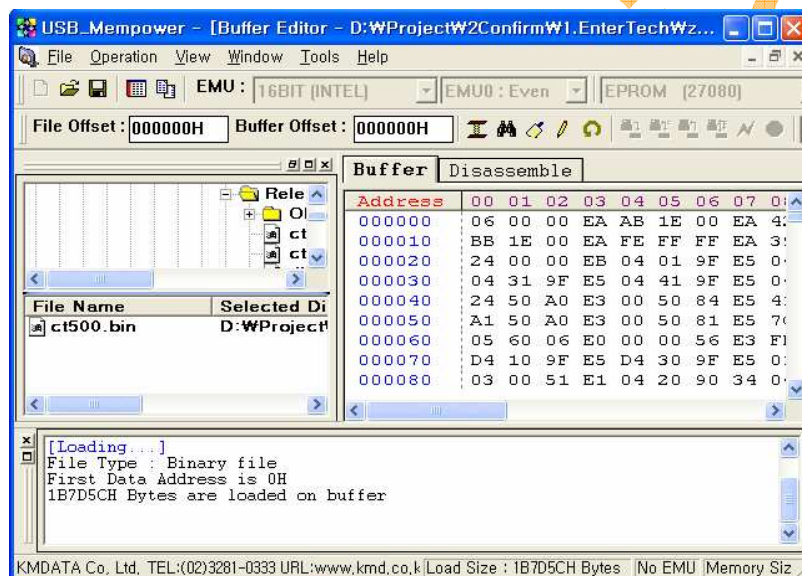


[Fig - 6] axd 를 사용한 디버깅 시

롬 에뮬레이터(External memory Select0)를 사용하는 경우는 릴리즈 모드에서 바이너리 파일을 생성하고 바이너리를 다운로드 하여 실행하면 된다.



[Fig - 7] 릴리즈 모드에서 실행 파일 생성 시



[Fig - 8] 롬에물레이터 사용 시

6. HOWTO Porting

상화에서 제공되는 CT-500 용 F/W 소스를 해당 어플리케이션에 맞게 포팅하기 위해서는 다음과 같이 하면 된다.

- 1) 메모리 맵과 스택 영역에 맞추어 **startup** 코드를 수정한다.

개발자의 특별한 요구 사항과 메모리 맵의 수정이 필요한 경우를 제외하고는 대개의 경우 상화 제공 **startup** 코드를 그대로 사용하면 된다.

main/startup/startup.s

- 2) 개발 어플리케이션에 맞게 소스를 작성한다.

main/mmu.c

main/app

응용프로그램을 개발 시 [Fig - 5]에서와 같이 **main/** 디렉토리 부분에서만 수정이 이루어지면 된다. 응용프로그램을 호출하는 **main()** 함수와 해당 응용프로그램 함수의 구현이 이루어지는 **app** 디렉토리가 그 부분이다.

6.1. Memory map

상화에서 제공되는 F/W 소스의 메모리 맵은 바이너리 실행 파일을 사용하는 ROM 기반일 때와 디버거를 사용하는 DDR 기반으로 각각의 메모리 맵은 다음과 같다.

- 1) 롬에올레이터 기반

: 롬에올레이터가 **external memory select0** 영역에, NOR 플래쉬가 **external memory select1** 영역에 있을 경우

- ROM 영역

- Code 영역 : 0x00000000 ~ 0x00200000

F/W 코드가 위치하는 영역

- DDR-SDRAM 영역

- Data 영역 : 0x60100000 ~ 0x60700000

전역변수 및 **static** 변수가 위치하는 영역

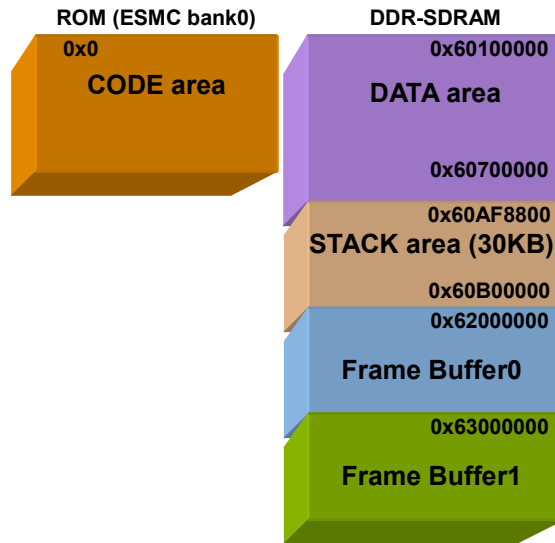
- Stack 영역 : 0x60AF8800 ~ 0x60B00000

Exception 모드(User/SVC/IRQ/FIQ/ABT/UND) 별 스택 영역

- Frame buffer0 : 0x62000000 ~ 0x62FFFFFF

Jpeg 디코딩을 위한 **frame buffer0** 영역

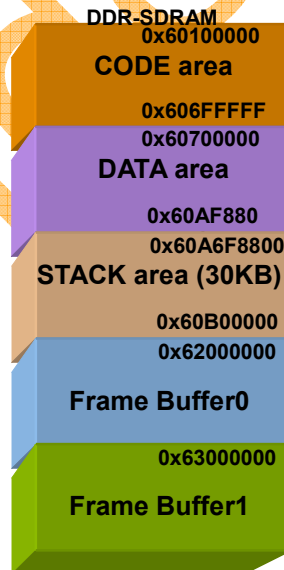
- Frame buffer1 : 0x63000000 ~ 0x63FFFFFF



[Fig - 9] ROM base memory map

2) DDR-SDRAM 기반

: A1000 과 같은 디버거로 DDR-SDRAM 영역을 사용, NOR 플래시가 external memory select1 영역에 있을 경우



[Fig - 10] DDR-SDRAM 기반 메모리 맵

6.2. System setting

개발하고자 하는 제품에 맞도록 syscfg.h의 시스템 설정을 수정하면 된다. 그러나 특별한 경우를 제외하고 대개의 경우 상화에서 제공하는 플랫폼의 시스템 설정을 그대로 사용하면 된다.

6.3. Source modification

1) 메모리 맵과 스택 영역에 맞추어 **startup** 코드 수정

개발하고자 하는 응용프로그램의 메모리 맵을 위한 수정이 필요한 경우를 제외하고는 **startup**과 메모리 맵은 수정하지 않아도 된다.

2) ROM 기반과 DDR-SDRAM 기반에 따라서 **mmu.c** 수정

- **MMU_Init()** 함수에서 다음 부분을 수정한다.

```
#if 1 // DDR-SDRAM base 일 경우 활성화  
MMU_SetMTT(0x62000000,0x63F00000,0x62000000,RW_CNB); // DDR 15MByte  
MMU_SetMTT(0x60100000,0x61F00000,0x60100000,RW_CNB); // DDR 48MByte  
  
MMU_SetMTT(0x00000000,0x00100000,0x60000000,RW_CNB); // DDR 1MByte  
#else // ROM base 일 경우 활성화  
MMU_SetMTT(0x62000000,0x63F00000,0x62000000,RW_NCNB); // DDR 32MByte  
MMU_SetMTT(0x60000000,0x61F00000,0x60000000,RW_CNB); // DDR 32MByte  
MMU_SetMTT(0x00000000,0x00200000,0x00000000,RW_CNB); // DDR 2MByte  
#endif
```

3) 응용 프로그램에 맞게 **main/app** 디렉토리에 소스를 작성

APPENDIX

A. ASSEMBLER DIRECTIVES

1) Unary Operators

- DEF - Usage -> :DEF:A {TRUE} if A is defined, otherwise {FALSE}
- NOT - Usage -> :NOT:A Bitwise complement of A
- LNOT - Usage -> :LNOT:A Logical complement of A

2) Logical operators

- OR - A:OR:B Bitwise OR of A and B
- EOR - A:EOR:B Bitwise Exclusive OR of A and B

3) Symbol definition directives

- GBLA - Declares a global arithmetic variable
- GBLL - Declares a global logical variable
- GBLS - Declares a global string variable
- SETA - Sets the value of a local or global arithmetic variable
- SETL - Sets the value of a local or global logical variable
- SETS - Sets the value of a local or global string variable

4) Data definition directives

- MAP - Sets the origin of a storage map to a specified address
 - MAP expr(,base-register} Address is expr + base-register's value
 - ^ is a synonym for MAP
- FIELD - Describes space within a storage map that has been defined using the MAP directive.
 - # is a synonym for FIELD
- LTORG - Literal pool(상수 공간)의 .org를 지정
- SPACE - Reserves a zeroed block memory
- DCD - Allocates one or more words(4Bytes) of memory
 - Defines the initial runtime contents of the memory

5) Miscellaneous directives

CODE16/CODE32

- Instructs the assembler to interpret subsequent instructions as 16/32bit instruction

6) Reporting directives

ASSERT- Generates an error message if a given assertion is false

OPT - Listing options

- 1: Turn on normal listing 2: off normal listing

- 4: Page throw 128: Turn off listing of macro expansions

Confidential

C. CACHE AND MMU INSTRUCTION

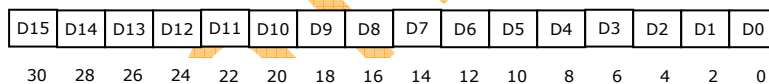
ARM926EJ-S 코어는 MMU control register(cp15:c1)의 비트 맵과 TLB flush register(cp15:c8), TLB lock down register(cp15:c10), domain-control register(cp15:c3), 고속 문맥 전환 확장 register(cp15:c13), 변환 테이블 베이스 주소(cp15:c2)를 사용하여 mcr 또는 mrc 명령어를 통해 MMU와 I/D-Cache를 제어한다.

명령	MCR 명령어	Remark
모든 TLB를 무효화	mcr p15, 0, Rd, c8, c7,0	ARM920T와 ARM926EJ-S 동일
라인으로 TLB 무효화	mcr p15, 0, Rd, c8, c7,1	
I TLB 무효화	mcr p15, 0, Rd, c8, c5,0	
라인으로 I TLB 무효화	mcr p15, 0, Rd, c8, c5,1	
D TLB 무효화	mcr p15, 0, Rd, c8, c6,0	
라인으로 D TLB 무효화	mcr p15, 0, Rd, c8, c6,1	

[Table - 1] TLB 플러시하기 위한 cp15:c8 명령어

명령	MCR 명령어	Remark
D TLB 락다운 읽기	mrc p15, 0, Rd, c10, c0,0	ARM920T와 ARM926EJ-S 동일
D TLB 락다운 쓰기	mcr p15, 0, Rd, c10, c0,1	
I TLB 락다운 읽기	mrc p15, 0, Rd, c10, c0,1	
I TLB 락다운 쓰기	mcr p15, 0, Rd, c10, c0,1	

[Table - 2] TLB 락다운 레지스터를 액세스하는 명령



[Fig - 11] 도메인 액세스 컨트롤 레지스터 cp15:c3 의 포맷

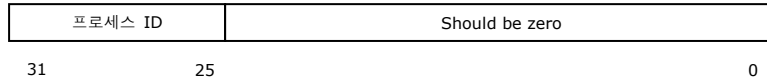


[Fig - 12] MMU 안에 위치한 cp15:c1 레지스터 컨트롤 비트

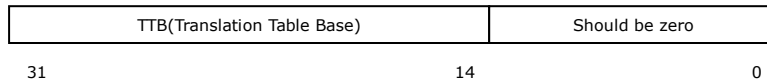
비트	약자	인에이블 가능	의미
0	M	MMU	0=disable, 1=enable
2	C	D-Cache	0=disable, 1=enable
3	W	Write Buffer	0=disable, 1=enable
8	S	System	
9	R	ROM	
12	I	I-Cache	0=disable, 1=enable

13	V	High vector table	0=0x000000 에 vector table 위치 1=0xffff0000 에 vector table 위치
----	---	-------------------	--

[Table - 3] MMU 제어 컨트롤 레지스터 cp15:c1 의 비트필드



[Fig - 13] 고속 문맥 전환 레지스터 cp15 레지스터 13



[Fig - 14] 변환 테이블 베이스 주소 cp15 레지스터

ARM920T, ARM926EJ-S cache의 경우 I/D cache로 구성되며, 세가지 정책(write policy, cache replacement, cache allocate policy)에 있어 동일하다.

1) 다음은 I/D cache의 flush/clean에 사용되는 cp15의 register 및 명령어이다.

기능	MCR 명령어	Remark
캐시 플러시	mcr p15, 0, Rd, c7, c7,0	ARM920T와 ARM926EJ-S 동일
데이터 캐시 플러시	mcr p15, 0, Rd, c7, c6,0	ARM920T와 ARM926EJ-S 동일
명령어 캐시 플러시	mcr p15, 0, Rd, c7, c5,0	ARM920T와 ARM926EJ-S 동일

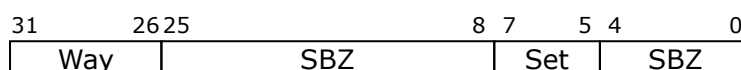
[Table - 4] Cache flush 명령어

2) 다음은 way와 세트 인덱스 주소 지정 방식으로 cp15:c7를 이용하여 D cache를 clean하는 명령어이다.

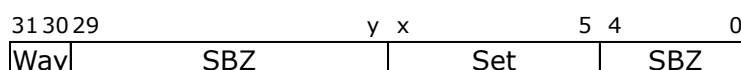
기능	MCR 명령어	Remark
명령어 캐시 라인 플러시	mcr p15, 0, Rd, c7, c5, 2	ARM926EJ-S(920T은 지원 안함)
데이터 캐시 라인 플러시	mcr p15, 0, Rd, c7, c6, 2	ARM926EJ-S(920T은 지원 안함)
데이터 캐시 라인 클린	mcr p15, 0, Rd, c7, c10, 2	ARM920T와 ARM926EJ-S 동일
데이터 캐시 라인 클린 및 플러시	mcr p15, 0, Rd, c7, c14, 2	ARM920T와 ARM926EJ-S 동일

[Table - 5] Way와 세트 인덱스 주소 지정 방식을 사용한 cache line clean & flush cp15:c7 명령어

ARM920T



ARM926EJ-S



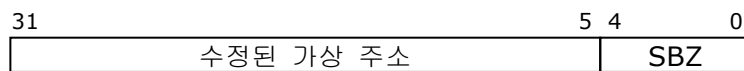
[Fig - 15] Way와 세트 인덱스 주소 지정 방식을 사용한 cache clean 시 cp15:c7 레지스터 Rd의 형식

3) 다음은 Cache의 일부분을 clean하고 flush하는 명령어이다. (가상 주소가 사용된다.)

기능	MCR 명령어	Remark
명령어 캐시 라인 플러시	mcr p15, 0, Rd, c7, c5, 1	ARM920T와 ARM926EJ-S 동일
데이터 캐시 라인 플러시	mcr p15, 0, Rd, c7, c6, 1	ARM920T와 ARM926EJ-S 동일
데이터 캐시 라인 클린	mcr p15, 0, Rd, c7, c10, 1	ARM920T와 ARM926EJ-S 동일
데이터 캐시 라인 클린 및 플러시	mcr p15, 0, Rd, c7, c14, 1	ARM920T와 ARM926EJ-S 동일

[Table - 6] 주메모리 안에서 위치를 참조하여 cache line clean 하고 flush 하는 명령어

ARM920T, ARM926EJ-S



[Fig - 16] 주메모리 안에서 원래 값으로 참조된 cache line 을 clean 하고 flush 할 때의 cp15:c7 레지스터 포맷

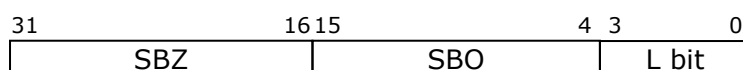
4) 락다운 된 cache 데이터/코드는 교체되지 않으며, cache가 flush되면 락다운 된 정보는 사라진다. Cache lock down 방법은 way addressing, lock down bit, 특수 allocate 명령어를 통한 방법이 있다. 특수 allocate 명령어를 통한 lock down은 xScale ARM을 위한 방법이며, way addressing 방법은 ARM920T와 ARM926EJ-S 동일하게 적용 가능하다. 반면 lock down bit 방법은 ARM926EJ-S(ARM920T 지원 안함)에 적용 가능한 방법이다.

ARM926EJ-S의 경우 way addressing과 lock down bit를 이용한 방법이 가능하나 여기서는 lock down bit 방법만을 기술하도록 한다.

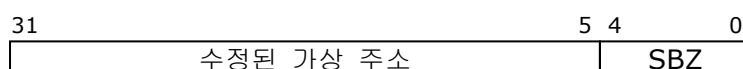
명령	MCR과 MRC 명령어
I 캐시 락 페이지 레지스터 읽기	mrc p15, 0, c9f, c9, c0, 1
D 캐시 락 페이지 레지스터 읽기	mrc p15, 0, c9f, c9, c0, 0
I 캐시 락 페이지 레지스터 쓰기	mcr p15, 0, c9f, c9, c0, 1
I 캐리 락 페이지 레지스터 쓰기	mcr p15, 0, c9f, c9, c0, 0
주소에서 코드 캐시 라인 읽기	mcr p15, 0, Rd, c7, c13, 1

[Table - 7] Lock down bit 를 사용하여 cache 를 lock 하기 위한 cp15:c9 명령어

ARM926EJ-S



ARM920T, ARM926EJ-S



[Fig - 17] Lock down bit 사용 cache lock 했을 때 cp15:c9 레지스터 포맷

[Fig-9]의 첫번째 그림이 I/D cache lock bit와 관련 된 것이며, 두 번째 그림이 가상 주소를 이용한 cache line lock 방법 시 레지스터 포맷이다.